

第五章：数据库完整性——AI 讲解

根本矛盾

数据库的初衷是让数据能被多人共享。但共享 + 多人修改 + 复杂的业务规则，就意味着数据随时可能被"搞坏"——可能是输错，也可能是不同表之间对不上。

整个第五章就解决一个问题：怎么保证数据库里的数据"对得上现实、也对得上自己"？

第四章讲的是"谁能进来"（安全性，管"门"）。第五章讲的是"进来的东西合不合格"（完整性，管"货"）。一扇门、一仓库，必须两件事都做好。

一、什么是数据完整性

数据完整性 = 正确性 + 相容性，两件事缺一不可。

维度	大白话	仓库类比
正确性	符合现实世界语义	货物本身合格——没有过期、没有损坏
相容性	同一对象在不同表中的数据符合逻辑	账本对得上——仓库账本和财务账本里同一笔货的描述一致

恍然大悟点：很多同学把"完整性"等同于"正确性"，漏掉了"相容性"。举个例子：学生表里写着"张三的年龄是 20 岁"，成绩表里写着"张三是个 1955 年出生的退休教授"——每一条单独看都"像"对的，但两张表合起来就矛盾。这就是相容性问题。

二、维护完整性的 3 个功能

数据库要维护完整性，必须有 3 道关卡，顺序不能反：

定义完整性约束条件	→	提供完整性检查方法	→	进行违约处理
(先写游戏规则)		(再装监控探头)		(违规就抓)

步骤	大白话	工厂类比
定义约束条件	告诉 DBMS"什么算对的数据"	收货标准：什么货能进、什么不能进
提供检查方法	DBMS 主动检查每次操作	X 光机：每件货都过一遍
违约处理	检查不通过时怎么办	违规就退货/拒收

恍然大悟点：为什么顺序不能反？

- 没定义约束就检查？检查器不知道什么算"对"，瞎忙。
 - 没检查就直接处理？没有触发点，处理无从下手。
- 考试常考的坑："以下哪个不是维护完整性的功能"——其实 3 个功能是成套的，缺一个机制就不完整。

三、实体完整性——"学号"的故事



3.1 主码是什么

主码 = 用来唯一标识一个实体的一列（或一组列），由数据库管理员人为指定。

比喻：学号。每个学生一个号、不能重复、不能为空。

- 不是身份证号（身份证号是天生的、由国家统一编码；主码是数据库管理员挑的，可以是学号、工号、甚至新加的 ID 列）

3.2 两个独立的检查

源材料特别强调：实体完整性有两个分开的检查步骤，不能合并。

检查	内容	怎么做
① 主码是否唯一	主码整体不能重复	插入或修改时查重
② 主码各分量是否非空	组成主码的每一个属性都不能为 NULL	即使是复合主码 (Sno, Cno)，Sno 和 Cno 也都不能空

恍然大悟点：为什么是"主码"而不是"任意 UNIQUE 列"？

- UNIQUE 列只是"恰好不重复"，但主码是用来标识实体的——实体在数据库里的"身份"必须明确。
 - 打个比方：公司里"姓名"可能 UNIQUE，但工号才能代表"这个员工"。换部门了名字没换、换手机号了工号也没换——工号才是身份的根。
 - 而且主码不能空（空就分不清"谁是谁"），UNIQUE 列允许 NULL——这就是本质区别。
- 违约处理：主码不唯一 / 主码有空 → 直接拒绝插入或修改，没有"商量"的余地。

四、参照完整性 —— 本章最核心的考点【重点】

4.1 外码是什么

外码 = 一张表（参照表/子表）里的一列，它的值对应另一张表（被参照表/父表）的主码。

比喻：订单表上的"客户编号"。订单属于哪个客户？订单上要写明。客户编号就是外码，对应客户表的主码。

注意：外码是实实在在的列值（如 'C001'），不是内存地址/指针。它是"引用"，不是"链接"。

4.2 4 种破坏情形（核心考点）

参照完整性可能被 4 种操作破坏，2 类发起方：

情形	发起方	破坏原因	违约处理
① 参照表插入元组	子表	新元组的外码值在父表里找不到	拒绝
② 参照表修改外码值	子表	改成的新外码值在父表里找不到	拒绝
③ 被参照表删除元组	父表	子表里还有引用该元组的记录（"悬空引用"）	拒绝 / 级联删除 / 置空（三选一）
④ 被参照表修改主码值	父表	子表里引用的主码值变了（"引用错位"）	拒绝 / 级联修改 / 置空（三选一）

4.3 速记口诀

"子表违拒绝，父表违规三选"



>

- 子表（参照表）的违规 = 插入/修改外码 → 外码指向不存在的东西，只能拒绝
- 父表（被参照表）的违规 = 删除/修改主码 → 父表主动变，可以连带处理子表（拒绝/级联/置空）

4.4 三种违约处理怎么选

策略	含义	适用场景	比喻
拒绝（NO ACTION）	不让操作	严格场景，不容许悬空	父亲改名字时儿子不许改
级联（CASCADE）	父表改/删，子表跟着改/删	强父子关系	父亲改名字时儿子姓氏一起改
置空（SET NULL）	父表改/删，子表外码置 NULL	弱引用，子表可独立存在	父亲改名字时儿子把姓氏留空

恍然大悟点：为什么"插入/修改外码"没有"级联"和"置空"？

- 级联是把"父的变动传给子"，子表这边在制造引用，根本没有"父"可以传。
- 置空是"父没了，子留空"，子表这边引用都没建立起来，怎么置空？
- 换句话说：级联和置空都是父表变动 → 子表跟随的链式反应；子表自己造引用失败时，没有"跟随对象"，只能"拒绝"。

五、用户定义完整性 —— "业务规则千变万化"

实体完整性和参照完整性是关系模型本身就要求的（来自数学）。但现实业务千变万化——年龄不能为负、性别只能是男女、男同学不能选妇产科学——这些规则靠用户自己定义。

用户定义完整性分两层：属性上和元组上。

5.1 属性上的约束条件（管"一列"）

约束	作用	例子
NOT NULL	该列不能为空	学号不能空
UNIQUE	该列不能重复	手机号唯一
CHECK	该列取值满足表达式	年龄 ≥ 0 且 ≤ 150

```
Sno CHAR(9) NOT NULL,  
Sname CHAR(20) UNIQUE,  
Sage INT CHECK (Sage >= 0 AND Sage <= 150),  
Sex CHAR(1) CHECK (Sex IN ('M', 'F'))
```

5.2 元组上的约束条件（管"一行多列"）

有些规则单独看每一列都满足，组合起来才出问题——这就是元组级 CHECK 存在的意义。

典型例子："男同学不能选修'妇产科学'课程"

- 光看 Sex = 'M'：没问题
- 光看 Cname = '妇产科学'：没问题
- 组合看："性别男 + 课程是妇产科学"：违规！

```
CHECK (NOT (Sex = 'M' AND Cname = '妇产科学'))
```

或者拆成逻辑等价形式：



5.3 属性级 vs 元组级 CHECK 对比

维度	属性级 CHECK	元组级 CHECK
作用范围	一列的取值范围	一行内多列之间的约束
出现位置	列定义后面	CREATE TABLE 语句中作为表级 CHECK
典型场景	年龄 ≥ 0	男同学不能选妇产科学
谁能用	单字段规则	跨字段规则 (必须元组级)

恍然大悟点：属性级 CHECK 和元组级 CHECK 看起来都是 CHECK，本质区别在哪？

- 属性级 CHECK 是“这一列的取值范围”——只看自己这一列。
- 元组级 CHECK 是“这一行的多个列之间要互相协调”——必须看一行里多个列的组合。

举个例子，光看 Sex='M'、光看 Cname='妇产科学' 都合法，但组合起来就有问题。只有元组级 CHECK 才能管这种“跨列”约束。

违约处理：用户定义完整性被破坏时，只有一种处理方式——拒绝。没有“级联”或“置空”这种选项——因为用户定义完整性约束是对单行/单列的硬性规定，不像参照完整性有“父表带子表”那种链式关系。

六、与第四章（安全性）的对比 —— 一管“门”，一管“货”

第四章和第五章是数据库保护机制的两条腿，解决的问题完全不同：

维度	第四章 安全性	第五章 完整性
根本矛盾	谁有资格用数据	数据本身对不对
管什么	“门”——谁能进、能干啥	“货”——进来的东西合不合格
核心机制	身份鉴别 + 授权 (GRANT/REVOKE)	约束条件 + 检查 + 违约处理
比喻	门锁 + 钥匙	入库标准 + X 光机 + 退货
触发时机	用户尝试访问时	用户尝试增删改时
失败处理	拒绝访问	拒绝操作 / 级联 / 置空

恍然大悟点：两者经常被混淆。区分的简单方法：

- 问“是谁”的问题 → 安全性 (这个用户有没有权限？)
- 问“是什么”的问题 → 完整性 (这个数据合不合法？)

易混淆对比总结

表 1：完整性的两个维度

维度	关注点	反例
正确性	单条数据符合现实	写了“年龄 = 200”
相容性	多表之间数据自治	学生表年龄 20，成绩表里生日是 1955

表 2：三大完整性总览



完整性	约束对象	来源	检查内容	违约处理
实体完整性	主码	关系模型本身	唯一 + 各分量非空	拒绝
参照完整性	外码	关系模型本身	外码值在父表中存在	拒绝 / 级联 / 置空
用户定义完整性	属性 / 元组	业务规则	满足 CHECK 等条件	拒绝

表 3：参照完整性的三种违约处理

策略	何时用	含义	适用情形
拒绝	严格场景	不让操作	父表删除/修改主码
级联删除	强父子关系	父删子跟着删	父表删除主码
级联修改	强父子关系	父改子跟着改	父表修改主码
置空	弱引用	父变子外码留空	父表删除/修改主码

表 4：属性级 vs 元组级 CHECK

维度	属性级 CHECK	元组级 CHECK
作用	一列的取值范围	一行内多列的相互约束
例子	年龄 ≥ 0	男同学不能选妇产科学
违约处理	拒绝	拒绝

逻辑主线

完整性的两个维度（正确性 + 相容性） ↓ 维护完整性的 3 个功能（定义 → 检查 → 处理） ↓ 三大完整性（按“约束对象”递进） ├ 实体完整性：主码（最小唯一标识） ─ 唯一性检查 ─ 主码各分量非空检查 → 违约处理：拒绝 ├ 参照完整性：外码（子表引用父表） ─ 4 种破坏情形（子表 2 种 + 父表 2 种） ─ 速记：子表违规拒绝，父表违规三选 ─ 3 种违约处理：拒绝 / 级联 / 置空 └ 用户定义完整性：业务规则 ├ 属性级：NOT NULL / UNIQUE / CHECK └ 元组级：跨字段 CHECK → 违约处理：拒绝 ↓ 与第四章（安全性）对比：一管“门”，一管“货”

整个第五章就一件事：让数据库里的数据既“对得上现实”（正确性），也“对得上自己”（相容性），并通过三大完整性机制把这件事自动化。

